

Отчет по лабораторной работе №14

Программирование в командном процессоре ОС UNIX.

Расширенное программирование

Виноградова Мария Андреевна

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
3	Выводы	13

Список иллюстраций

2.1	Пишем командный файл	7
2.2	Запускаем командный файл в двух терминалах	8
2.3	Смотрим на результат работы	8
2.4	Пишем командный файл	9
2.5	Тестируем командный файл с несуществующей командой	10
2.6	Тестируем командный файл с существующей командой	11
2.7	Пишем командный файл	12
2.8	Смотрим на результат выполнения командного файла	12

Список таблиц

1 Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов

2 Выполнение лабораторной работы

1. Написать командный файл, реализующий упрощённый механизм семафоров. Командный файл должен в течение некоторого времени t_1 дожидаться освобождения ресурса, выдавая об этом сообщение, а дождавшись его освобождения, использовать его в течение некоторого времени $t_2 < t_1$, также выдавая информацию о том, что ресурс используется соответствующим командным файлом (процессом). (рис. 2.1).

The screenshot shows a Gedit editor window with the title bar "1.sh (~/.lab14) - gedit". The window contains a shell script with the following content:

```
1 #!/bin/bash
2
3 if [ $# -ne 2 ]; then
4     echo "Использование: $0 <t1> <t2>"
5     exit 1
6 fi
7
8 t1=$1
9 t2=$2
10
11 semaphore_file="semaphore.lock"
12
13 touch $semaphore_file
14
15 function access_resource {
16     while ! ln $semaphore_file $0.lock 2>/dev/null; do
17         echo "Ресурс занят, ожидание освобождения..."
18         sleep $t1
19     done
20
21     echo "Ресурс освобожден, начало использования на $t2
22 секунд"
23     sleep $t2
24     echo "Ресурс освобожден, использование завершено"
25     rm $0.lock
26 }
27
28 access_resource
```

The status bar at the bottom indicates "sh", "Ширина табуляции: 8", "Ln 13, Col 22", and "INS".

Рис. 2.1: Пишем командный файл

1.2. Запустить командный файл в одном виртуальном терминале в фоновом режиме, перенаправив его вывод в другой (> /dev/tty#, где # — номер терминала куда перенаправляется вывод), в котором также запущен этот файл, но не фоновом, а в привилегированном режиме.

Начало работы (рис. 2.2).

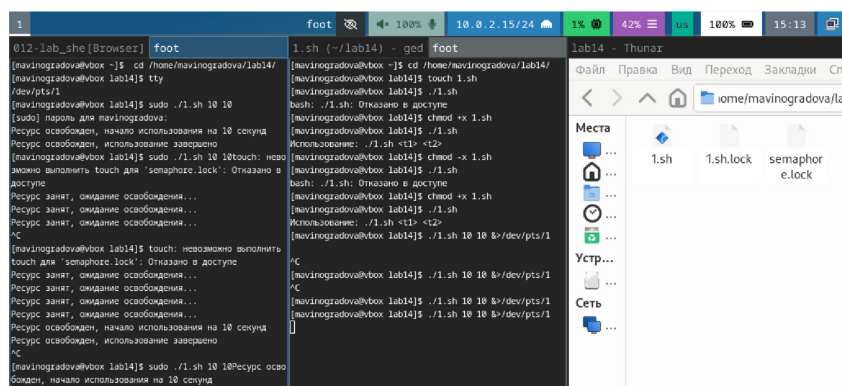


Рис. 2.2: Запускаем командный файл в двух терминалах

Завершение работы: (рис. 2.3).

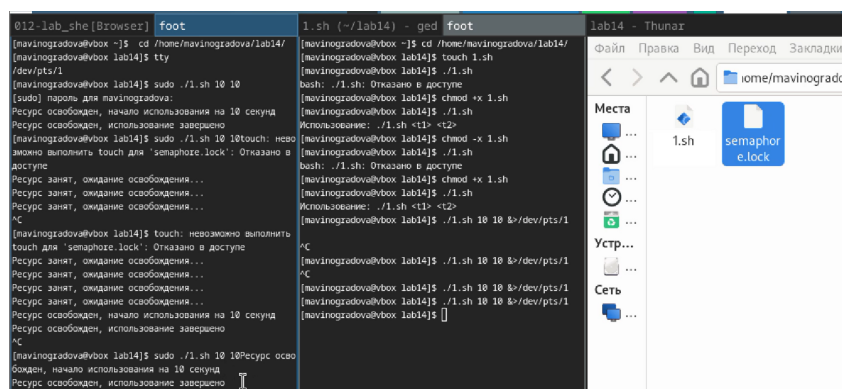
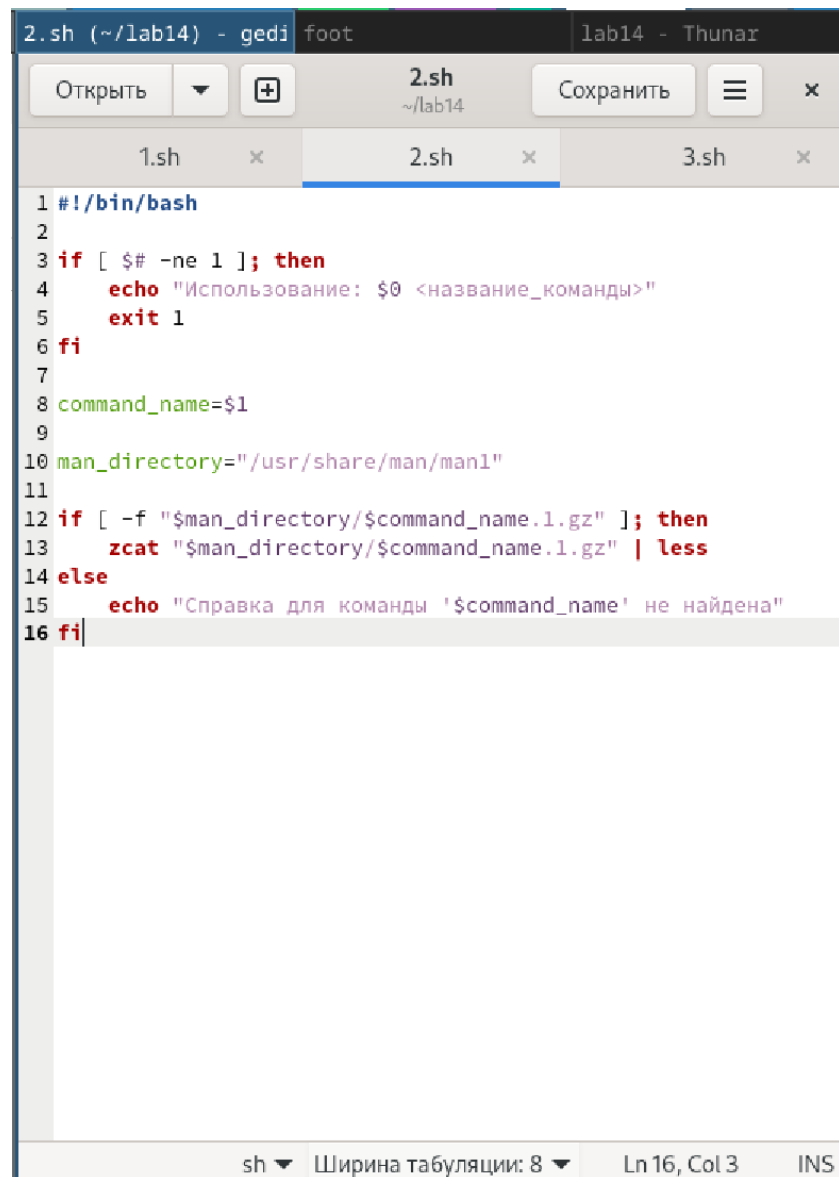


Рис. 2.3: Смотрим на результат работы

2. Реализовать команду `man` с помощью командного файла. Изучите содержимое каталога `/usr/share/man/man1`. В нем находятся архивы текстовых файлов, содержащих справку по большинству установленных в системе программ и команд. Каждый архив можно открыть командой `less` сразу же просмотрев содержимое справки. Командный файл должен получать в виде аргумента командной строки название команды и в виде результата выдавать справку об этой команде или сообщение об отсутствии справки, если соответствующего файла нет в каталоге `man1` (рис. 2.4).



```
2.sh (~/.lab14) - gedit foot lab14 - Thunar
Открыть 2.sh Сохранить
1.sh 2.sh 3.sh
1 #!/bin/bash
2
3 if [ $# -ne 1 ]; then
4     echo "Использование: $0 <название_команды>"
5     exit 1
6 fi
7
8 command_name=$1
9
10 man_directory="/usr/share/man/man1"
11
12 if [ -f "$man_directory/$command_name.1.gz" ]; then
13     zcat "$man_directory/$command_name.1.gz" | less
14 else
15     echo "Справка для команды '$command_name' не найдена"
16 fi
```

sh Ширина табуляции: 8 Ln 16, Col 3 INS

Рис. 2.4: Пишем командный файл

Работа командного файла с несуществующей командой: (рис. 2.5).

```
[mavinogradova@vbox lab14]$ ./2.sh
Использование: ./2.sh <название_команды>
[mavinogradova@vbox lab14]$ ./2.sh kko
Справка для команды 'kko' не найдена
[mavinogradova@vbox lab14]$ ./2.sh ls
```

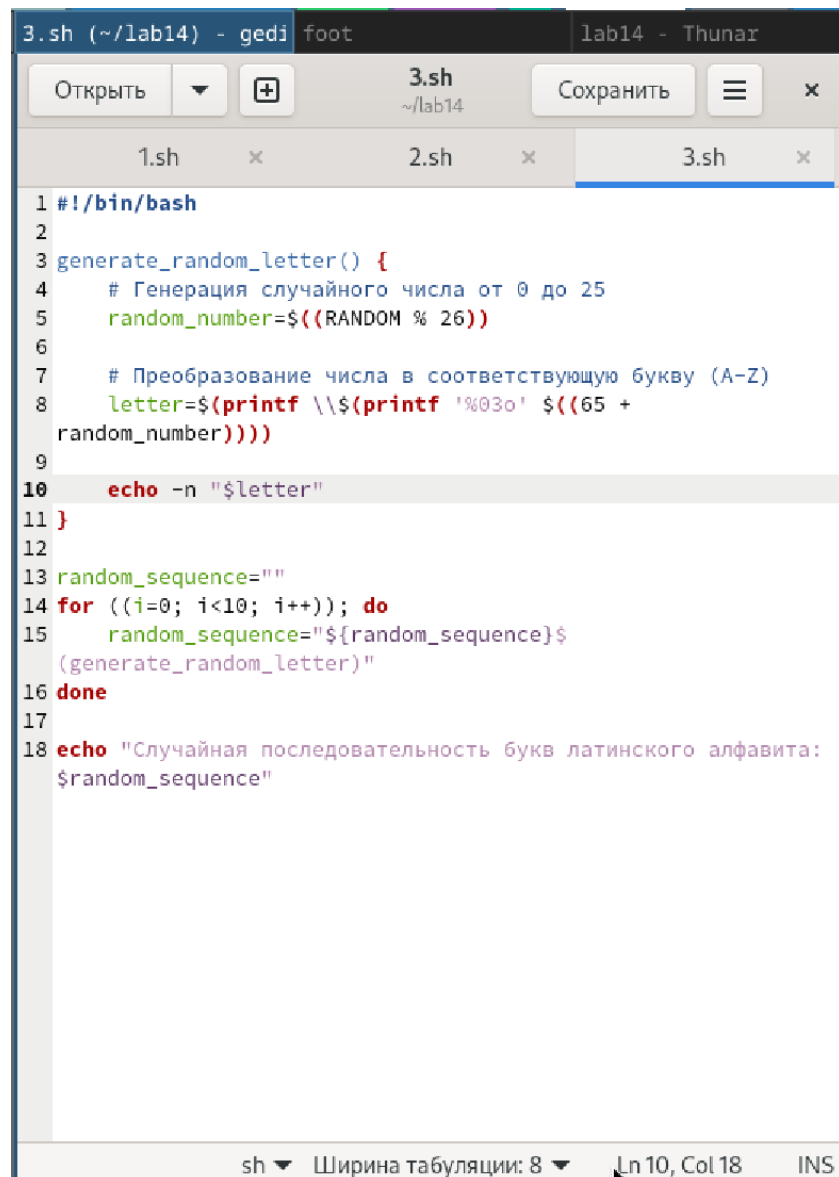
Рис. 2.5: Тестируем командный файл с несуществующей командой

Работа командного файла с существующей командой: (рис. 2.6).

```
3.sh (~/.lab14) - gedi foot lab14 - Thunar
.\ " DO NOT MODIFY THIS FILE! It was generated by help2man 1.48.5.
.TH LS "1" "November 2024" "GNU coreutils 9.5" "User Commands"
.SH NAME
ls \- list directory contents
.SH SYNOPSIS
.B ls
[\fI\,OPTION\ \fR]... [\fI\,FILE\ \fR]...
.SH DESCRIPTION
.\ " Add any additional description here
.PP
List information about the FILEs (the current directory by default).
Sort entries alphabetically if none of \fB\ -cftuvSUX\ \fR nor \fB\ -\ -sort\ \fR is specifie
d.
.PP
Mandatory arguments to long options are mandatory for short options too.
.TP
\fB\ -a\ \fR, \fB\ -\ -all\ \fR
do not ignore entries starting with .
.TP
\fB\ -A\ \fR, \fB\ -\ -almost\ -all\ \fR
do not list implied . and ..
.TP
\fB\ -\ -author\ \fR
with \fB\ -l\ \fR, print the author of each file
.TP
\fB\ -b\ \fR, \fB\ -\ -escape\ \fR
print C\ -style escapes for nongraphic characters
.TP
\fB\ -\ -block\ -size\ \fR=\fI\,SIZE\ \fR
with \fB\ -l\ \fR, scale sizes by SIZE when printing them;
e.g., '\fB\ -\ -block\ -size=M\ \fR'; see SIZE format below
.TP
\fB\ -B\ \fR, \fB\ -\ -ignore\ -backups\ \fR
do not list implied entries ending with ~
.TP
\fB\ -c\ \fR
with \fB\ -lt\ \fR: sort by, and show, ctime (time of last
change of file status information);
with \fB\ -l\ \fR: show ctime and sort by name;
otherwise: sort by ctime, newest first
.TP
\fB\ -C\ \fR
list entries by columns
```

Рис. 2.6: Тестируем командный файл с существующей командой

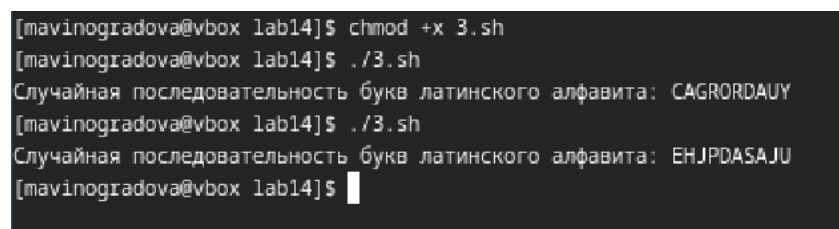
- Используя встроенную переменную \$RANDOM, напишите командный файл, генерирующий случайную последовательность букв латинского алфавита. (рис. 2.7).



```
1 #!/bin/bash
2
3 generate_random_letter() {
4     # Генерация случайного числа от 0 до 25
5     random_number=$((RANDOM % 26))
6
7     # Преобразование числа в соответствующую букву (A-Z)
8     letter=$(printf \\$(printf '%03o' $((65 +
9 random_number))))
10    echo -n "$letter"
11 }
12
13 random_sequence=""
14 for ((i=0; i<10; i++)); do
15     random_sequence="${random_sequence}${
16 (generate_random_letter)}"
17 done
18 echo "Случайная последовательность букв латинского алфавита:
19 $random_sequence"
```

Рис. 2.7: Пишем командный файл

Работа командного файла: (рис. 2.8).



```
[mavinogradova@vbox lab14]$ chmod +x 3.sh
[mavinogradova@vbox lab14]$ ./3.sh
Случайная последовательность букв латинского алфавита: CAGRORDAUU
[mavinogradova@vbox lab14]$ ./3.sh
Случайная последовательность букв латинского алфавита: EHJPDASAJU
[mavinogradova@vbox lab14]$
```

Рис. 2.8: Смотрим на результат выполнения командного файла

3 Выводы

В ходе работы освоено создание сложных shell-скриптов с использованием условий и циклов, что позволяет эффективно автоматизировать задачи в UNIX-системах. Приобретённые навыки значительно расширяют возможности работы в командной строке.